

ANN Project — Smart Fitness Calories Prediction System

An Artificial Neural Network (ANN) built with TensorFlow/Keras that predicts whether an online shopping session will end in a purchase (the Revenue field), using the **Online Shoppers Purchasing Intention** dataset. The project covers data loading, label encoding, a train/test split, feature scaling, model training, and evaluation.

Note: The notebook's markdown title ("Smart Fitness Calories Prediction System") doesn't match the dataset or task actually used (*online_shoppers_intention.csv*, predicting Revenue). This README documents what the code actually does — you may want to rename the notebook/heading to match, or swap in the intended fitness dataset.

Overview

Item	Value
Task	Binary classification — predict Revenue (purchase: True/False)
Framework	TensorFlow / Keras
Dataset	online_shoppers_intention.csv
Dataset size	12,330 rows x 18 columns
Features used	17 (Revenue is the target)
Train / Test split	80% / 20% (random_state=42)
Training accuracy	38.08%
Testing accuracy	36.01%

Project Structure

```
.
├── ANN_Project.ipynb      # Main notebook (data prep -> training -> evaluation)
├── README.docx
└── data/
    └── online_shoppers_intention.csv
```

Note: The dataset CSV is not included in this repo. Update the file path in the notebook to point to your local copy of *online_shoppers_intention.csv*.

Requirements

```
pip install pandas numpy tensorflow scikit-learn matplotlib
```

Pipeline

1. Data Loading

The dataset is read with pandas from a local CSV. It contains 12,330 sessions with 18 columns describing browsing behaviour (page counts/durations, bounce/exit rates, page values, month, OS, browser, region, traffic type, visitor type, weekend flag) and the target column, Revenue (whether the session ended in a purchase).

2. Label Encoding

Four categorical / boolean columns are converted to numeric labels with scikit-learn's LabelEncoder:

- Month
- VisitorType
- Weekend
- Revenue (the target)

3. Feature / Target Split

Revenue is separated out as the target (y); the remaining 17 columns become the feature matrix (x).

4. Train / Test Split

The data is split 80/20 using train_test_split with random_state=42.

5. Feature Scaling

A StandardScaler is fit on the training features and applied to both splits, producing x_train_scale and x_test_scale.

6. Model Architecture

```
model = keras.Sequential([
    keras.layers.Dense(100, input_shape=(17,)), activation='relu'),
    keras.layers.Dense(50, activation='relu'),
    keras.layers.Dense(10, activation='relu'),
    keras.layers.Dense(10, activation='sigmoid')    # output layer
])
model.compile(optimizer='adam',
              loss='mse',
              metrics=['accuracy'])
model.fit(x_train, y_train, epochs=20)
```

The model is trained for 20 epochs on the **unscaled** training data (x_train, y_train), not the scaled arrays produced in the previous step.

7. Results

Metric	Value
Training Accuracy	38.08%
Testing Accuracy	36.01%

The author's note in the notebook: *"In my ANN project, I have accuracy on given dataset with epochs is 20 because the dataset rows is high (15k)."*

Usage

1. Clone the repo and open the notebook:

```
git clone <your-repo-url>
cd <your-repo>
jupyter notebook ANN_Project.ipynb
```

2. Update the CSV path in the data-loading cell to point to your local dataset.

3. Run all cells to preprocess the data, train the model, and view accuracy.

Possible Improvements

The current results (~36-38% accuracy) are well below what this dataset typically supports (Revenue is imbalanced at roughly 85% False / 15% True, so a naive baseline already scores ~85%). A few likely causes and fixes:

- Output layer mismatch: the final layer is `Dense(10, activation='sigmoid')` for a binary target — it should be `Dense(1, activation='sigmoid')`.
- Loss function: 'mse' is used for a classification task — switch to 'binary_crossentropy', which pairs correctly with a sigmoid output.
- Unused scaling: `x_train_scale` / `x_test_scale` are computed but the model is fit on the raw, unscaled `x_train` / `y_train` — train on the scaled arrays instead.
- Class imbalance: consider `class_weight`, oversampling (e.g. SMOTE), or stratified splitting to handle the ~85/15 imbalance in Revenue.
- Validation tracking: add a `validation_split` or `validation_data` argument to `model.fit` to monitor overfitting across epochs.
- Early stopping / regularization: add `EarlyStopping` and/or `Dropout` layers once the loss and output layer are corrected.
- Evaluation metrics: accuracy alone is misleading on imbalanced data — add precision, recall, F1, and a confusion matrix.